

Proyecto de Software

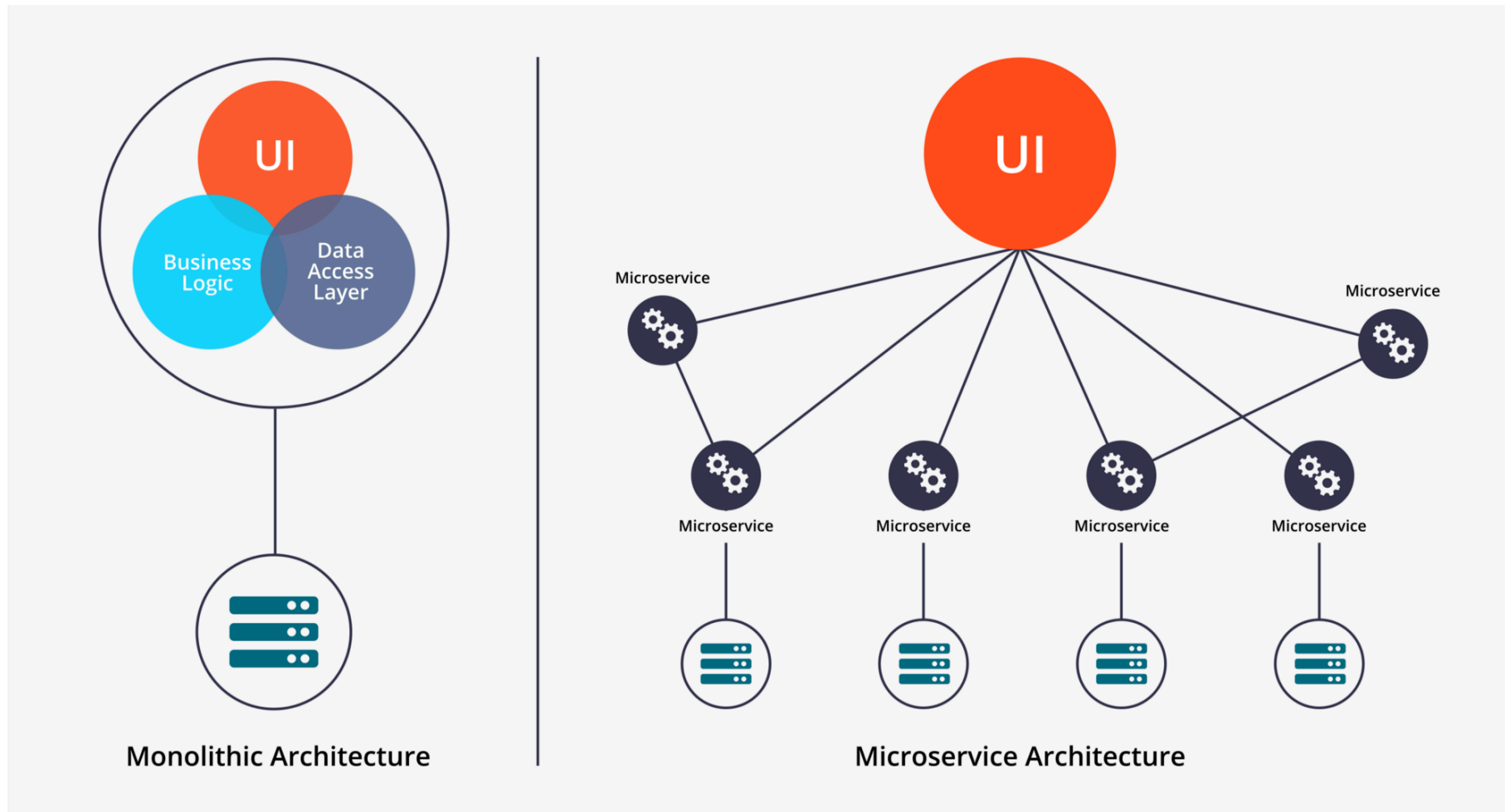
- Microservicios
- Frameworks JS
- Intro Vue

Arquitectura de Microservicios

Microservicios

- Las aplicaciones se **dividen en sus componentes más pequeños**, independientes entre sí.
- A diferencia del enfoque tradicional y monolítico de las aplicaciones, en el que todo se encuentra en una **única pieza**.
- Los microservicios funcionan en conjunto para llevar a cabo las **mismas tareas** que la aplicación monolítica.
- Los microservicios **facilitan la escalabilidad** de todo el sistema, se despliegan según se vayan necesitando.
- Pueden tener **distintas tecnologías** entre sí.
- Al ser más pequeños, son **más simples de mantener** y actualizar.

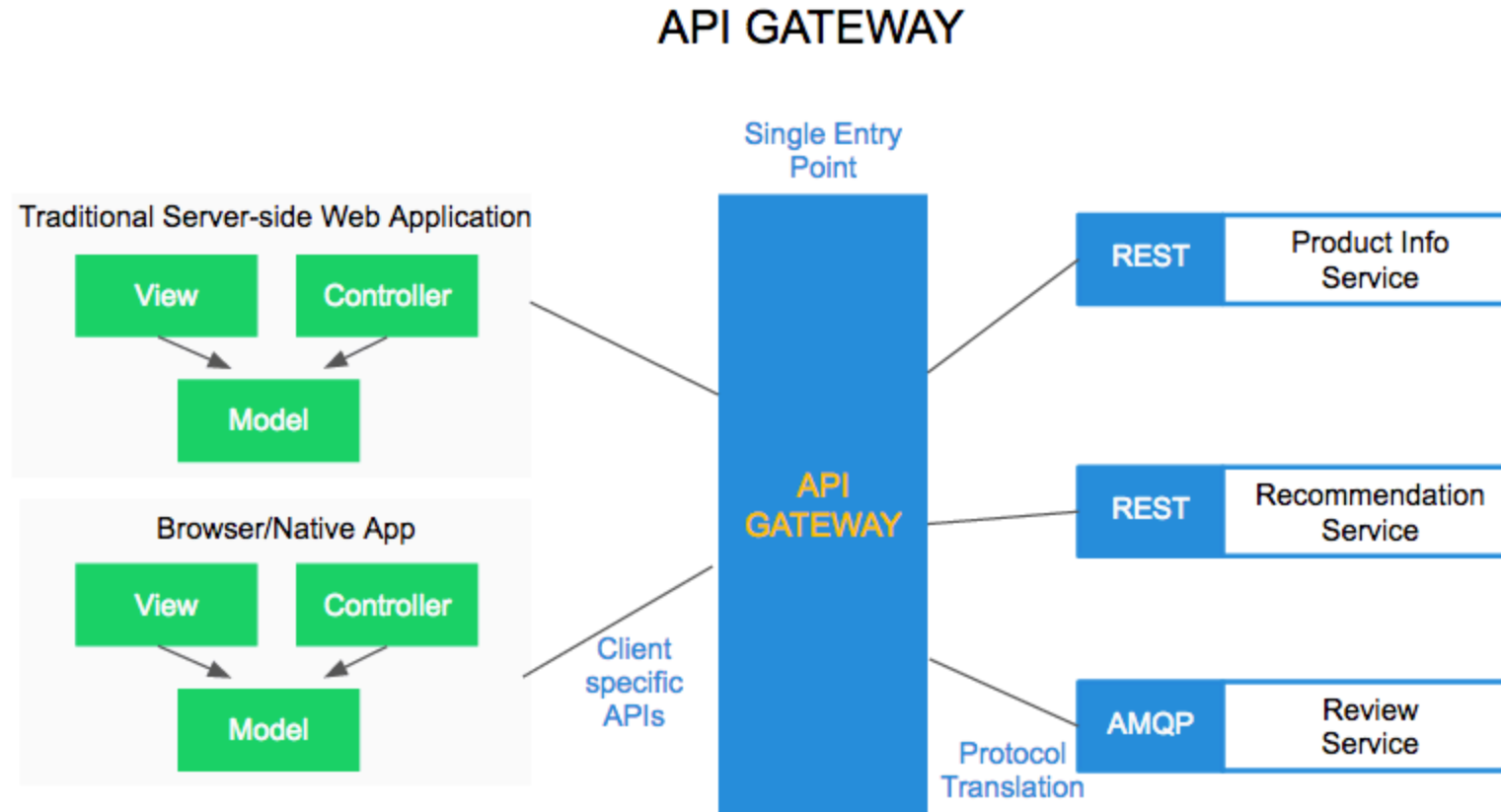
Arquitectura Monolítica vs Arquitectura de Microservicios



¿Cómo comunico un servicio con otro?

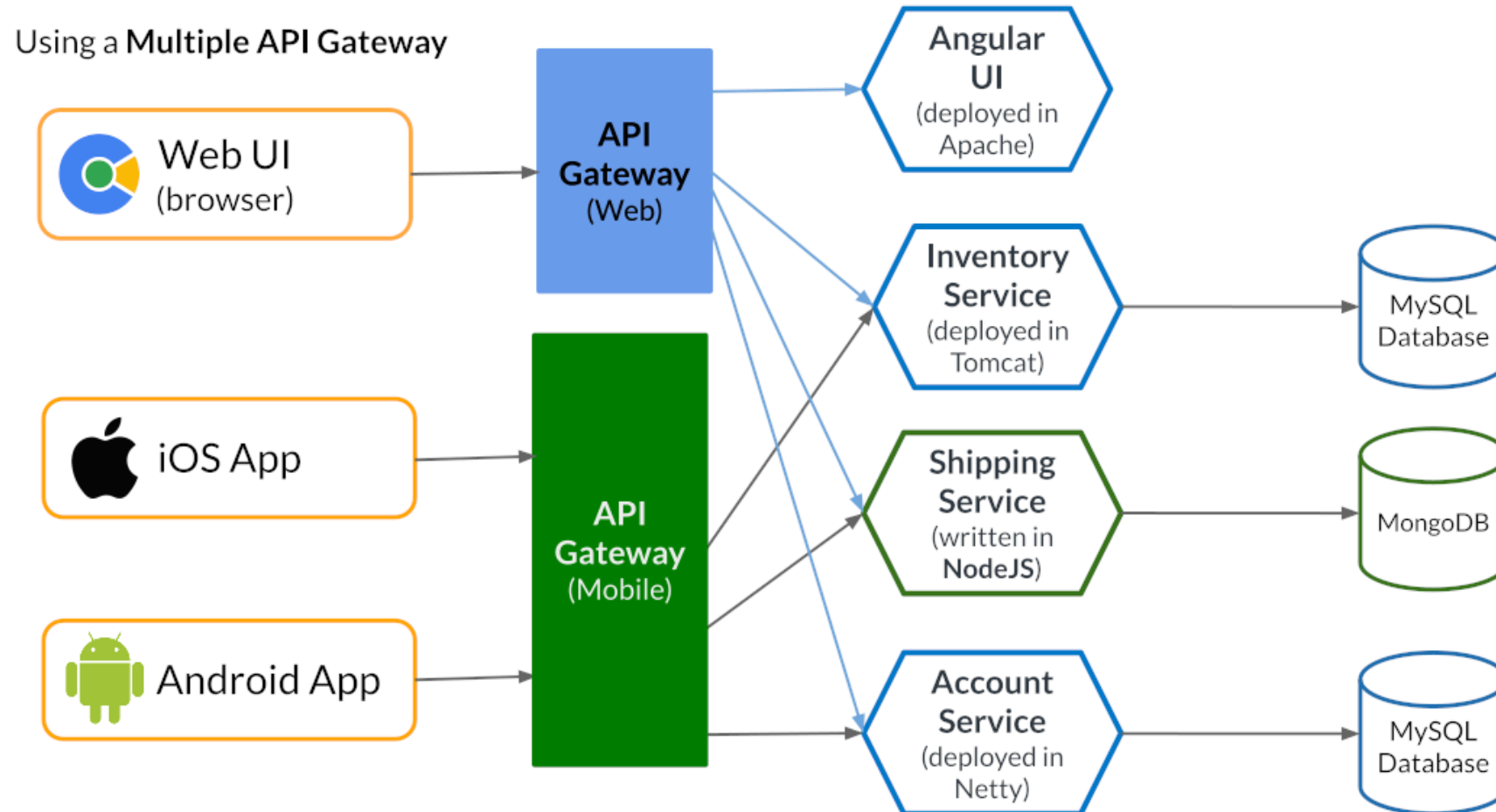
- Actualmente la opción más utilizada es mediante **APIs HTTP/REST** con JSON.
- Incluso puede centralizarse la comunicación utilizando un **API Gateway**.
- En dicho API Gateway puede implementarse una **capa de seguridad**, que ante una petición verifique si el cliente tiene permisos de acceso.

Microservicios: API Gateway

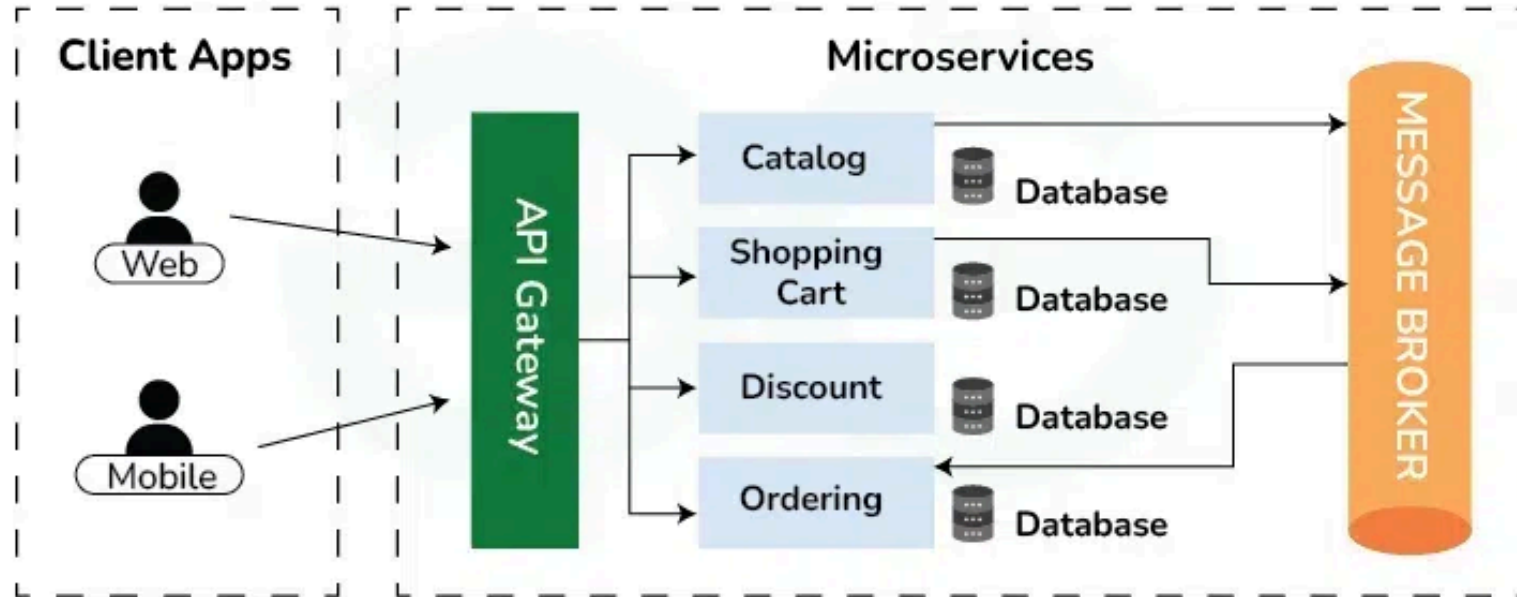


microservices.io

Microservicios: Múltiples API Gateway



Microservicios: Message Broker



Microservices Architecture



Top 6 message queues for distributed architectures

Para seguir leyendo de Microservicios

- AWS: <https://aws.amazon.com/es/microservices/>
- Redhat: <https://www.redhat.com/es/topics/microservices>
- Video en español: <https://www.youtube.com/watch?v=9R2hFwIPGnQ>

Frameworks JS

¿Librería o framework?

- Trabajar con **JS pelado** es complejo. Por eso nació **jQuery** (2006): una **librería** que simplifica el DOM y AJAX.
- **Librería**: nuestro código **la llama** cuando la necesita (jQuery, Axios).
- **Framework**: define la estructura de la aplicación y **llama a nuestro código** (Angular, Vue).
- jQuery sigue estando **muy** extendido, pero las aplicaciones web implementan **cada vez más funcionalidades y complejidad**, y una librería de atajos se queda corta.

Demasiadas librerías y frameworks JS

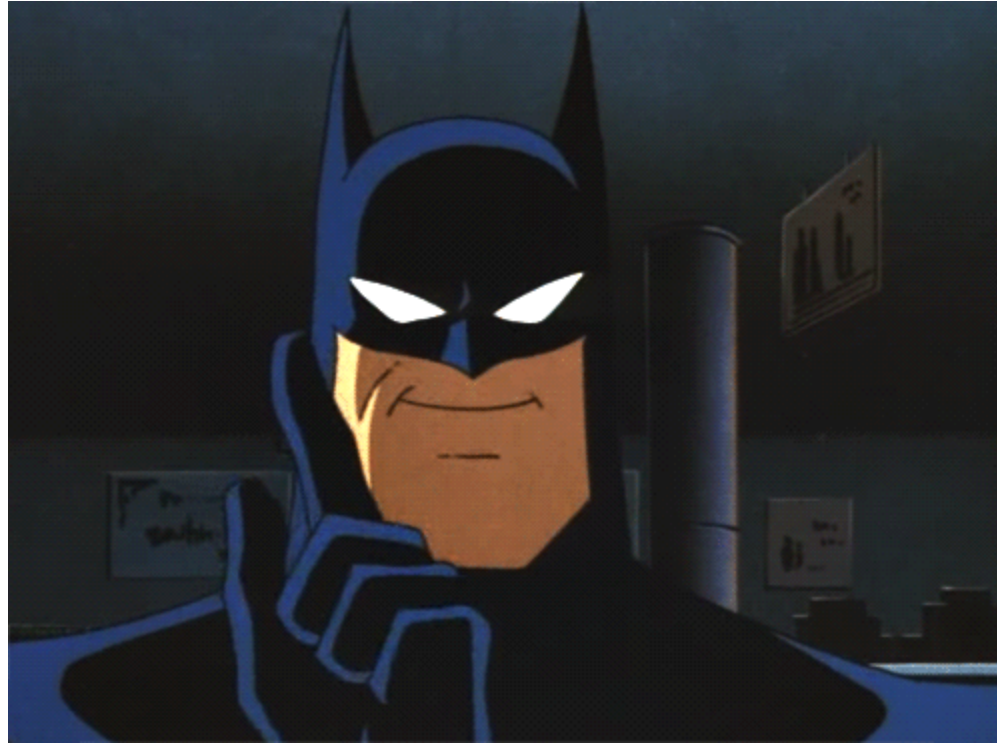
Backbone **Vue** Ember **React** Astro Knockout **Angular** htmx
Mithril **Svelte** Next.js **jQuery** Qwik Preact **Solid** Dojo Nuxt
Alpine.js Prototype **Lit** Remix AngularJS SvelteKit Meteor

- **Librerías:** jQuery, Preact, Lit...
- **Frameworks:** Angular, Vue, Svelte, Solid...
- **Meta-frameworks** (construidos sobre un framework): Next.js, Nuxt, SvelteKit...
- Lista (incompleta) en [Wikipedia](#).

¿Por qué existen los frameworks JS?

Será porque ...

- Están basados en componentes.
- Tienen una comunidad sólida.
- Tienen muchas bibliotecas y componentes de terceros para manejar diferentes tareas.
- Tienen extensiones de navegador que ayudan a depurar.
- Son buenos para crear aplicaciones SPA (single page applications).
- Poseen una buena interfaz de línea de comandos (CLI) para facilitar el desarrollo.
- Realizan optimizaciones para mejorar el rendimiento.



La verdadera razón:



KEEPING THE UI
IN SYNC
WITH THE STATE
IS HARD

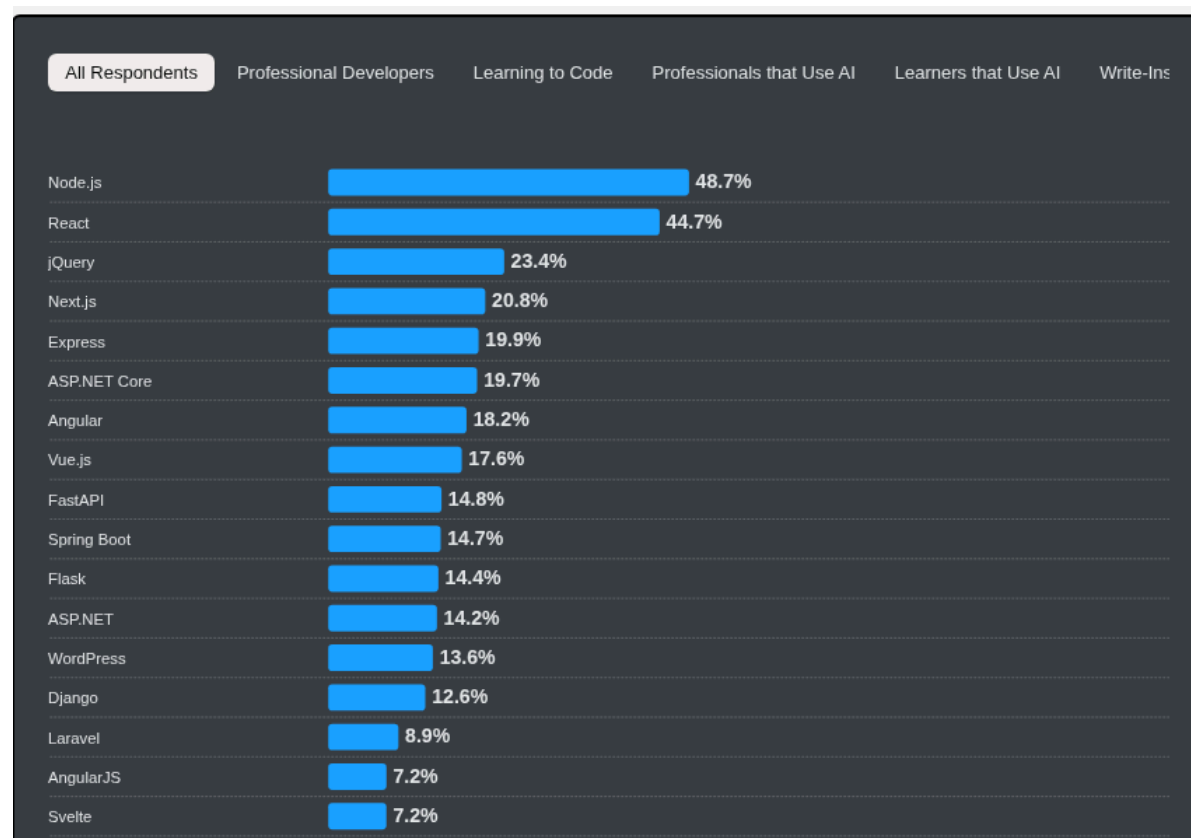
- Cada framework lo resuelve a su manera: **Virtual DOM**, **signals**, compilación... (lo vemos con Vue).
- [Artículo Medium](#)

Angular vs. React vs. Vue

	Angular	React	Vue
Tipo	Framework completo	Librería de UI	Framework progresivo
Mantenido por	Google	Meta	Evan You + equipo
Vistas	Templates HTML + TypeScript	JSX	Templates HTML + JS/TS
Actualización del DOM	Detección de cambios / signals	Virtual DOM	Reactividad + Virtual DOM
Ruteo y estado	Incluidos	De terceros	Oficiales (Vue Router, Pinia)
Curva de aprendizaje	Alta	Media	Suave
Uso (SO 2025)	18,2 %	44,7 %	17,6 %

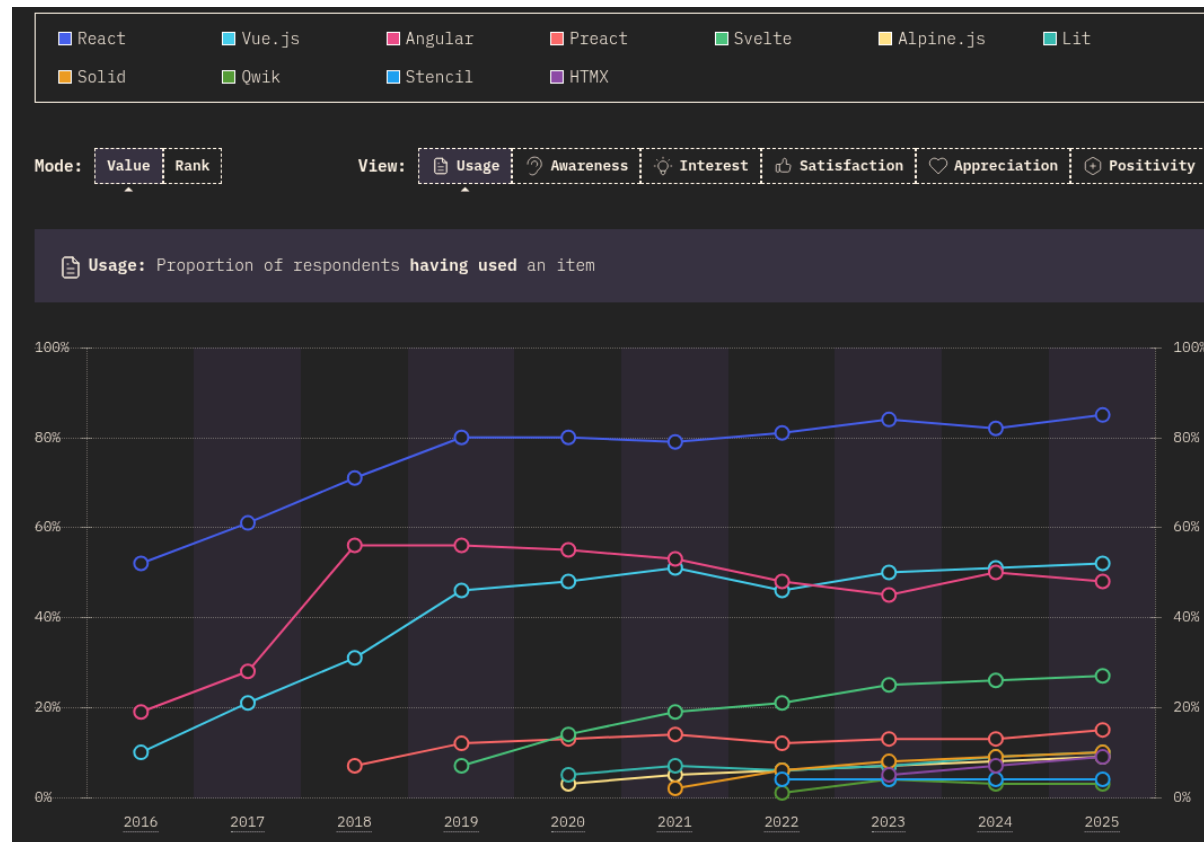
¿Qué se usa? Stack Overflow 2025

- Frameworks y tecnologías web más usados: mezcla front-end (React, Vue), back-end (Express, Flask, Django) y librerías (jQuery). Ver [encuesta](#).

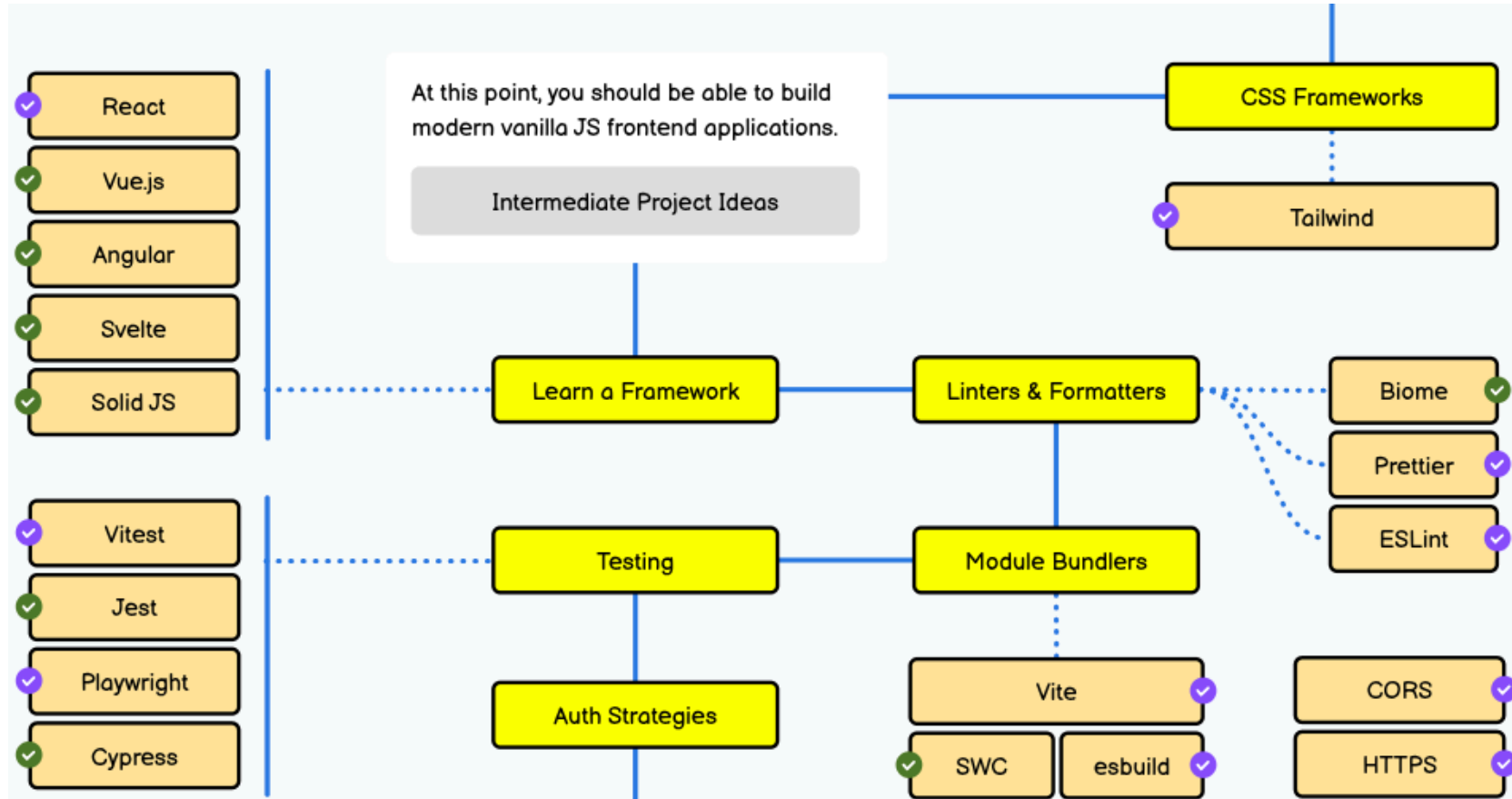


¿Qué se usa? State of JS 2025

- Proporción de personas que usaron cada framework front-end, según [State of JavaScript 2025](#).



¿Dónde encaja un framework? Front-end roadmap



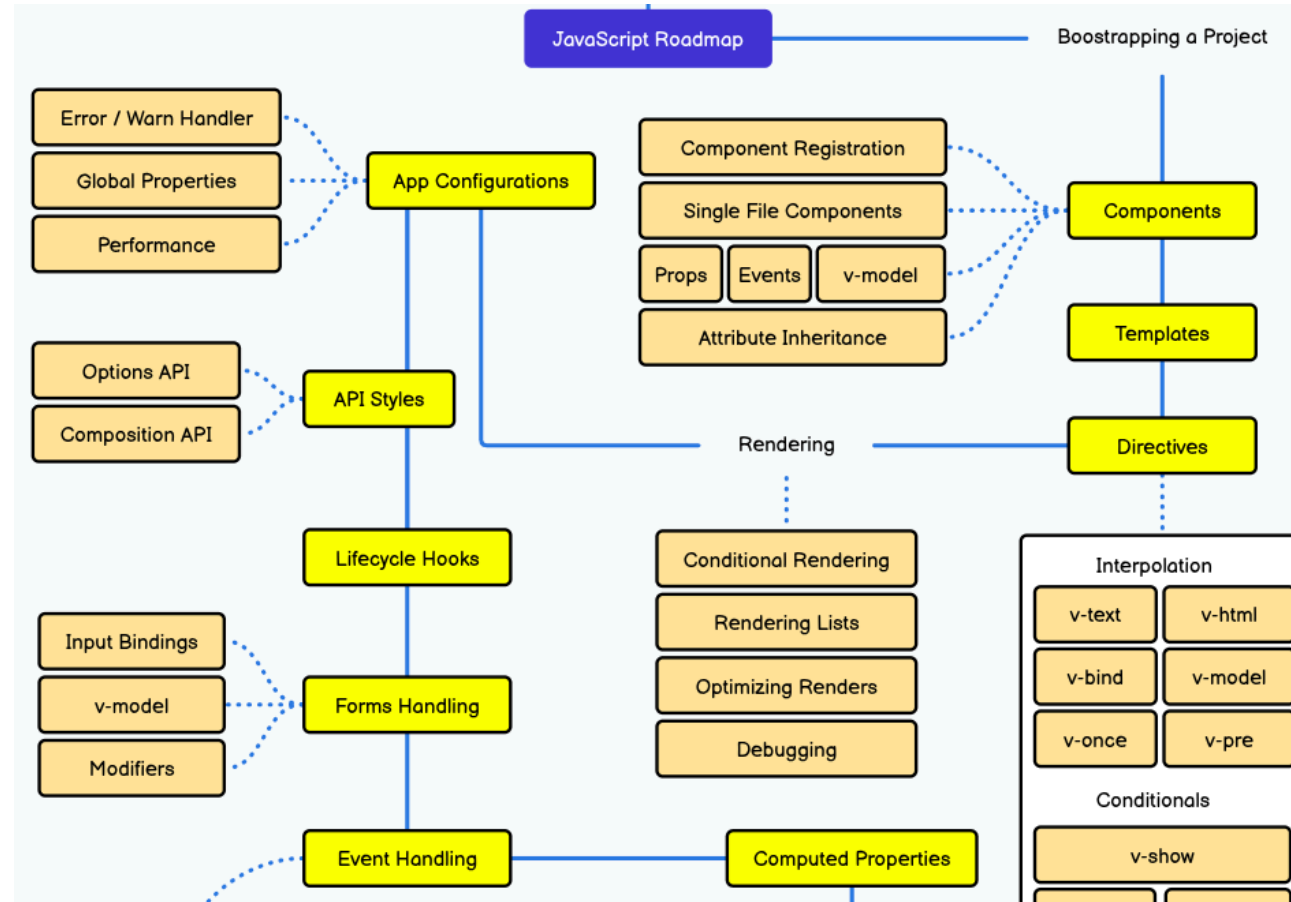
- Después de HTML, CSS, JS y Git. Ver roadmap completo en roadmap.sh/frontend.

Para seguir leyendo: JS Frameworks

- [Entendiendo los frameworks JS \(MDN\)](#)
- [Reactividad en JS](#)
- [Reactividad en profundidad \(Vue\)](#)
- [Benchmark de frameworks JS](#)

Vue.js

Vue.js roadmap: lo que vamos a ver



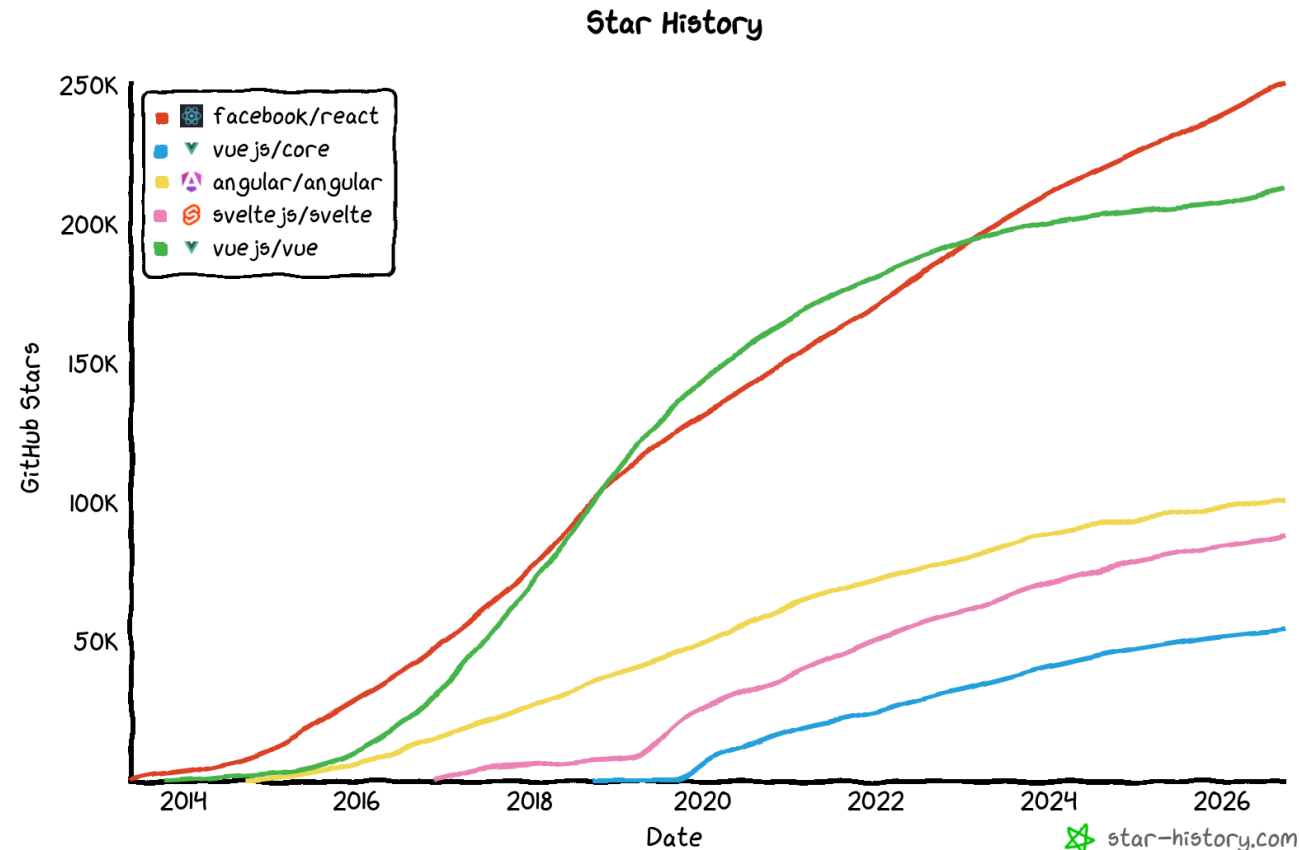
- Roadmap completo en roadmap.sh/vue.

Características

- Vue.js es un framework de JavaScript para front-end.
- **Curva de aprendizaje suave:** usa templates HTML con JavaScript.
- **Buen rendimiento:** un compilador optimiza los templates. Ver [benchmark](#).
- Vue.js es un framework **progresivo**. Tiene la facilidad para usarlo y adaptarlo a proyectos tanto grandes como pequeños.
- Ha tenido un gran crecimiento y tiene una comunidad muy activa.

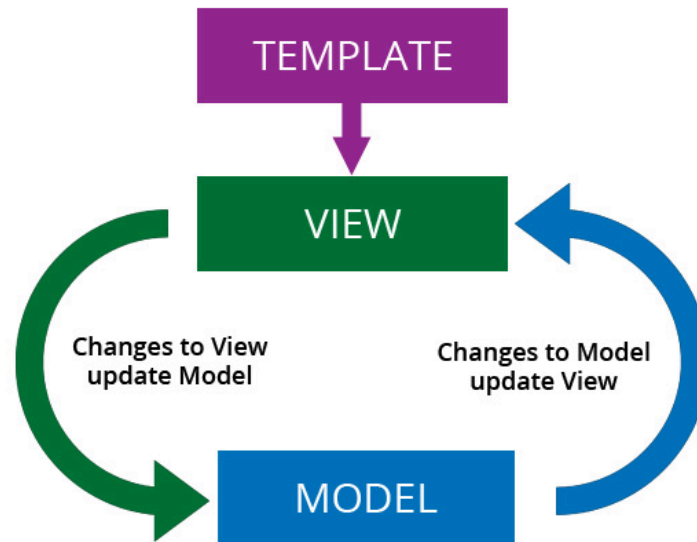
Comunidad de Vue en GitHub

- Estrellas según star-history.com. Ver la [colección de frameworks](#).
- Vue 2 está en **vuejs/vue** y Vue 3 en **vuejs/core**: sus estrellas están repartidas.



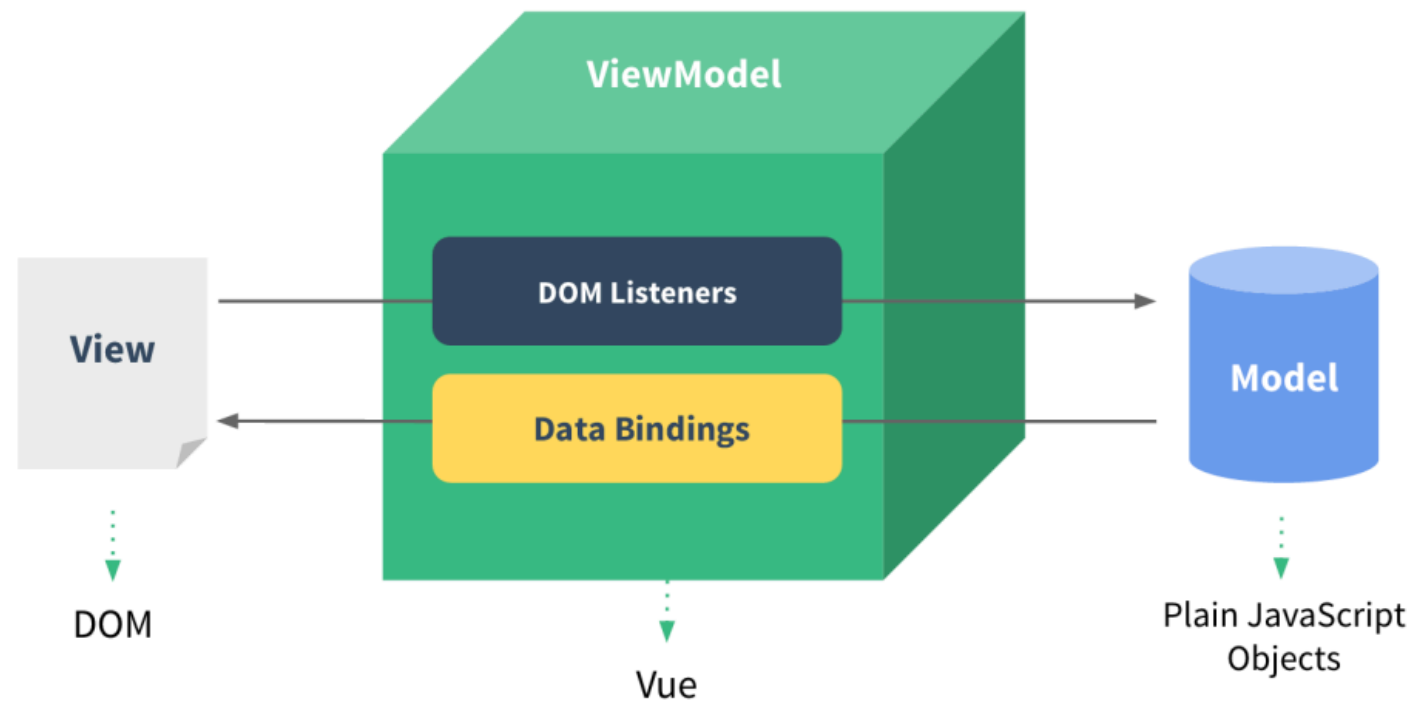
Reactividad y two-way data-binding

- Es un framework **reactivo**: cuando cambian los datos, Vue actualiza la vista automáticamente.
- En formularios, `v-model` agrega el camino inverso: lo que se escribe en un input actualiza los datos (**two-way data-binding**).



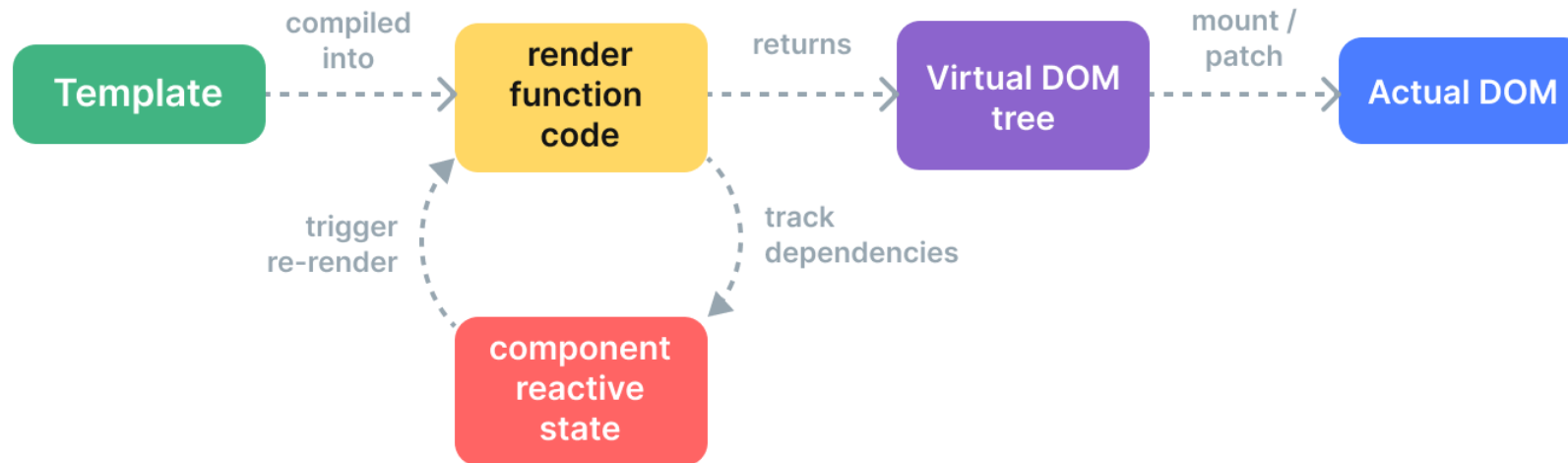
Patrón Model-View-ViewModel (MVVM)

- **Model:** los datos, objetos JS planos. **View:** el DOM. **ViewModel:** la instancia de Vue, que los conecta con *data bindings* y *DOM listeners*.



¿Cómo llega un cambio al DOM? Virtual DOM

- Modificar el DOM real es costoso. Vue (como React) trabaja sobre un **Virtual DOM**: una representación liviana del DOM hecha con objetos JS.
- El template se compila a una **función de render**, que devuelve el Virtual DOM. Cuando cambia el estado reactivo, se vuelve a ejecutar y Vue aplica al DOM real sólo las diferencias (*patch*).



Virtual DOM: paso a paso

- Template: `<div id="app"><h2>{{ message }}</h2><p>Proyecto</p></div>` . Ejecutamos `vue_app.message = 'Chau' :`

Virtual DOM antes

```
{ type: 'div', children: [
  { type: 'h2',
    children: 'Hola Vue!!!' },
  { type: 'p',
    children: 'Proyecto' }
]}
```

Virtual DOM después

```
{ type: 'div', children: [
  { type: 'h2',
    children: 'Chau' },      ← cambió
  { type: 'p',
    children: 'Proyecto' }  ← igual
}]
```

- El **diff** compara ambos árboles y el **patch** aplica un único cambio en el DOM real: `h2.textContent = 'Chau' .`
- El compilador de templates marca las partes estáticas (el `<p>`) para que el *diff* las saltee. Ver [mecanismo de renderizado](#).

Integración y herramientas

- Vue.js está más enfocado hacia la vista, y **puede ser implementado en el HTML de cualquier proyecto web** sin requerir cambios drásticos.
- Existe la extensión **Vue DevTools** para navegadores (también como plugin de Vite): devtools.vuejs.org. Nos asiste en el desarrollo.
- Vue 3 soporta los navegadores modernos con soporte nativo de **ES2016** (no soporta IE11). Ver [FAQ oficial](#) y [Versiones JS](#).

Instalación VueJS:

- Descargando el js e incluyéndolo directamente en un tag `<script>` .
- Linkear directamente desde una **CDN** (Content Delivery Network).

Vue 3 (Versión actual):

```
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
```

Vue 2 (Versión LTS - **finalizó el 31/12/2023**) - NO UTILIZAR:

```
<script src="https://cdn.jsdelivr.net/npm/vue@2"></script>
```

Instalación VueJS:

- Instalar Vue via **npm** (manejador de paquetes por defecto para Node.js):

```
npm create vue@latest
```



Se utiliza un setup guiado basado en [Vite](#) una herramienta para construir código web moderno.

```
cd NOMBRE_PROYECTO  
npm install  
npm run dev
```

- Es la forma que vamos a usar en el **proyecto**.
- Node.js versión requerida **^22.18.0 || >=24.12.0**.

Incluyéndolo de una CDN:

Veamos [hello_world](#). Los datos y DOM están ahora relacionados utilizando `{{}}`. Modifiquemos `vue_app.message`.

```
<title>Mi primera aplicaci&ocute;n Vue</title>
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
</head>
<body>
  <div id="app"> <h2>{{ message }}</h2> </div>
  <script>
    const HelloVueApp = {
      data() {
        return {
          message: 'Hola Vue!!!'
        }
      }
    }
    vue_app = Vue.createApp(HelloVueApp).mount('#app')
  </script>
```

Ejemplo Two way data-binding:

- Hace la relación bidireccional entre un input y los datos de la aplicación Vue.
- Veamos `v-model`.

```
<div id="two-way-binding">
  <p>{{ message }}</p>
  <input v-model="message">
</div>
<script>
  const App = {
    data() {
      return {
        message: 'Bienvenidos a Proyecto!!'
      }
    }
  }
  vue_app = Vue.createApp(App).mount('#two-way-binding')
</script>
```

¿Dudas?

Seguimos con las directivas de Vue...

Referencias Vue

- Guía oficial Vue: <https://vuejs.org/guide/introduction.html>